



TECHNICAL AUDIT & GAP ANALYSIS

Production Readiness & Engineering Maturity

End-to-end evaluation of the VERDICT platform across 6 enterprise software development pillars: Architecture, Data Hygiene, Automated Testing, Security, CI/CD, and Observability.

TARGET REPOSITORY

KATRIXBEE/verdict (main)

AUDIT DATE

26 August 2026

AUDITOR ROLE

Principal Architect & DevOps Lead

TARGET STACK

Next.js 15 App Router · TypeScript · PostgreSQL

AUDIT METHODOLOGY

NIST SP 800-53 / 12-Factor App / CIS

SDLC STATUS

Pre-Production Hardening Phase

1. Engineering Maturity Scorecard

The codebase was assessed against enterprise production-grade engineering criteria across six core software lifecycle dimensions. The scorecard summarizes current implementation completeness and assigned maturity grades:

SDLC PILLAR	STATUS	GRADE	ARCHITECTURAL ASSESSMENT
1. Architecture & Specs	PARTIAL	C	High-level README exists; lacks formal PRD, C4 architecture diagrams, and OpenAPI 3.0 contract.
2. Data Layer Hygiene	PARTIAL	C+	Comprehensive DDL schema with RLS and FKs exists; runtime currently operates from static JSON & SQLite rather than live transactional PostgreSQL.
3. Automated Testing	MISSING	D-	Strict TypeScript passes (<code>tsc --noEmit</code>), but **0 unit tests, 0 integration tests, and 0 E2E tests** exist. 7-parameter civic math is untested.
4. Security & Defense	IMPLEMENTED	A-	Zod mutation parsing, SSRF domain matching, Edge rate limiting, dynamic nonce CSP, HSTS preload, and sanitized error boundaries are fully operational.
5. CI/CD & DevOps	MISSING	F	Zero automated GitHub Actions workflows, no Dockerfile, no container compose, and no staging/production environment orchestration.
6. Observability & Telemetry	PARTIAL	D+	Opaque error envelopes prevent info leaks, but lacks structured JSON logging (Pino), APM error tracking (Sentry), and health check endpoints (<code>/api/health</code>).

Executive Architectural Summary The platform demonstrates excellent application security hygiene and clean TypeScript structure. However, it currently operates with the operational posture of an advanced prototype. Production readiness requires establishing automated CI/CD testing, connecting a live PostgreSQL database, and configuring APM telemetry.

2. Top Missing Architectural Artifacts

The following seven missing components represent the critical technical gap separating the current codebase from an enterprise production deployment:

1. Automated CI/CD Pipeline

[.github/workflows/ci.yml](#)

Impact: Pull requests can merge breaking changes or syntax errors directly into production without automated validation.

Requirement: GitHub Actions running `lint`, `tsc --noEmit`, unit test suites, and production build checks on all PRs.

2. Civic Scoring & API Unit Test Harness

[tests/unit/verdict-score.test.ts](#)

Impact: The 7-parameter civic scoring formula (attendance, criminal deductions, asset growth, education, party switches, MPLADS) has 0 automated tests.

Requirement: Vitest suite with 100% branch coverage on scoring formulas and storage locking deduplication logic.

3. Multi-Stage Containerization

[Dockerfile & docker-compose.yml](#)

Impact: Deployment is tightly coupled to cloud serverless hosting and cannot run portably on Kubernetes, AWS ECS, or private infrastructure.

Requirement: Lightweight multi-stage Dockerfile utilizing Next.js standalone output and non-root Alpine runtime.

4. Liveness & Readiness Probes

[src/app/api/health/route.ts](#)

Impact: Cloud orchestrators and load balancers cannot evaluate application health, memory usage, or storage connectivity before routing traffic.

Requirement: `GET /api/health` endpoint returning JSON health indicators and uptime metadata.

5. Structured Logging & Correlation IDs

[src/lib/logger.ts](#) (Pino)

Impact: Plain `console.error` calls lack request IDs, timestamps, and log levels, impeding debugging in log aggregation tools.

Requirement: High-performance JSON logger (Pino) with correlation IDs propagated from middleware.

6. Centralized Environment Schema

[src/env.mjs](#)

Impact: Missing runtime secrets fail at invocation time rather than throwing a clear descriptive error during application initialization.

Requirement: Zod-based environment variable validation schema executed at startup.

7. OpenAPI 3.0 API Specification

[docs/openapi.yaml](#)

Impact: Third-party civic developers and frontend components lack a machine-readable schema for consuming API endpoints.

Requirement: OpenAPI 3.0 documentation covering all 8 public REST routes.

3. Production Readiness Checklist

Detailed verification of standard artifacts, operational safeguards, and configurations required for enterprise production stability:

Pillar 1: Specifications & Architecture Documentation

- ✓ High-level product overview and civic domain description ([README.md](#))
- ✓ Project coding standards, design tokens, and architectural principles ([AGENTS.md](#))
- ✗ Formal Product Requirements Document (PRD) with user personas and acceptance criteria
- ✗ Technical Requirements Document (TRD) defining throughput, latency, and concurrency SLAs
- ✗ System architecture diagrams (C4 Context, Container, and Component models)
- ✗ OpenAPI 3.0 / Swagger schema specification for all 8 REST endpoints

Pillar 2: Data Layer & State Hygiene

- ✓ Version-controlled schema migration file ([supabase/migrations/20260820000000_verdict_schema.sql](#))
- ✓ Explicit foreign keys, check constraints, and indexes defined in DDL
- ✓ Row-Level Security (RLS) policies documented for multi-tenant table access
- ✓ Cross-process file locking ([proper-lockfile](#)) and atomic writes for local storage
- ✗ Direct application runtime connection to live transactional PostgreSQL / Supabase cluster
- ✗ Automated database migration runner (e.g., Supabase CLI / Drizzle / Prisma)
- ✗ Connection pooling (PgBouncer / Supabase Supavisor) configured for serverless scaling

Pillar 3: Automated Quality Assurance & Testing Suite

- ✓ Strict TypeScript configuration (`"strict": true` , `"noImplicitAny"`) with 0 errors
- ✓ Framework static analysis rules configured ([eslint-config-next](#))
- ✗ Unit test runner installed and configured (Vitest / Jest)
- ✗ Automated unit tests for VERDICT 7-parameter scoring engine ([src/lib/verdict-score-calc.ts](#))
- ✗ Integration tests for API routes ([/api/ratings](#) , [/api/search](#) , [/api/proxy-image](#))
- ✗ End-to-End browser test suite (Playwright / Cypress) covering critical voter flows
- ✗ Pre-commit lint and format hooks (Husky + lint-staged)

Pillar 4: Security & Defense-in-Depth

- [✓] Schema input validation on mutations with Zod (`/api/ratings`)
- [✓] SSRF-hardened image proxy with exact/dot-prefixed domain whitelist & 5MB stream cap
- [✓] Session and authentication infrastructure initialized via NextAuth.js
- [✓] Edge Middleware sliding-window rate limiting (IP-based with spoofing mitigation)
- [✓] Dynamic Content Security Policy (CSP) with per-request cryptographic nonces
- [✓] HTTP security headers (HSTS 2-year preload, `X-Frame-Options: DENY`, `nosniff`)
- [✓] Sensitive user-generated storage excluded from version control (`.gitignore`)
- [✓] Web crawler access controls configured (`public/robots.txt`)

Pillar 5: DevOps, CI/CD & Build Pipelines

- [✓] Reproducible production build command (`npm run build`) generating 576 static/dynamic paths
- [X] GitHub Actions CI workflow executing `lint`, `tsc`, and automated tests on PRs
- [X] Multi-stage production `Dockerfile` with non-root runtime user
- [X] Local container orchestration (`docker-compose.yml` for Next.js + PostgreSQL + Redis)
- [X] Formal Staging vs. Production deployment environment partitioning

Pillar 6: Observability, Logging & Error Handling

- [✓] Opaque error handling across all API routes preventing stack trace leakage
- [✓] Standardized error contract shape (`{ error: { code, message } }`)
- [X] Production Health & Readiness probe endpoints (`GET /api/health`)
- [X] Structured JSON logging library (Pino / Winston) with correlation IDs
- [X] Application Performance Monitoring (APM) and crash reporting (Sentry / Datadog)

4. Step-by-Step Production Roadmap

A prioritized, 5-phase engineering roadmap designed to systematically bridge the gap from current implementation to enterprise production standards:

PHASE 1 • P0 • WEEK 1

CI/CD AUTOMATION & QUALITY ASSURANCE HARNESS

1. **Configure GitHub Actions CI:** Create `.github/workflows/ci.yml` running `npm run lint`, `npx tsc --noEmit`, and build verification on all PR branches.
2. **Install Vitest Unit Testing:** Add `vitest` and write unit tests for `verdict-score-calc.ts` with 100% branch coverage across all 7 civic parameters.
3. **Configure Pre-commit Hooks:** Install `husky` and `lint-staged` to enforce typecheck and linting on every commit.

PHASE 2 • P0 • WEEK 2

LIVE DATABASE CONNECTION & MIGRATION AUTOMATION

1. **Connect Supabase/PostgreSQL Client:** Migrate data loading from static JSON to live PostgreSQL queries using `@supabase/supabase-js` or Drizzle ORM.
2. **Apply DDL Migrations:** Execute `20260820000000_verdict_schema.sql` on staging/production environments with automated migration tracking.
3. **Transactional Rating Mutations:** Transition `addCitizenRating` from file persistence to SQL `INSERT` with Row-Level Security policies.

PHASE 3 • P1 • WEEK 3

OBSERVABILITY, ERROR TRACKING & HEALTH CHECKS

1. **Implement Health Probe:** Create `src/app/api/health/route.ts` returning system uptime, memory RSS, and database latency metrics.
2. **Integrate Sentry APM:** Install `@sentry/nextjs` for real-time error tracking and performance profiling across server and client boundaries.
3. **Structured JSON Logging:** Implement `src/lib/logger.ts` using Pino, attaching `x-request-id` from edge middleware to all server logs.

CONTAINERIZATION & DISTRIBUTED RATE LIMITING

1. **Multi-Stage Dockerfile:** Configure Next.js `output: 'standalone'` and build a production container image (< 150MB).
2. **Environment Validation:** Create `src/env.mjs` using Zod to validate all required environment variables at application startup.
3. **Redis-Backed Rate Limiter:** Connect middleware rate limiting to Upstash Redis for persistent tracking across multi-pod clusters.

OPENAPI GOVERNANCE & DEVELOPER PLATFORM

1. **Publish OpenAPI 3.0 Contract:** Create `docs/openapi.yaml` documenting all public API endpoints and schema models.
2. **Mount Interactive Docs:** Serve Swagger UI or Scalar documentation at `/api-docs` for civic developers and journalists.